Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский ядерный университет „МИФИ“»

Отчёт по лабораторной работе № 2
по дисциплине «Криптография»
Изучение алгоритма RSA

Вариант № 2

Выполнил: студент 1-го курса магистратуры

Гафт Илья Маратович

Преподаватель: Епишкина Анна Васильевна

Москва — 2026

1. Цель работы

Освоить механизм шифрования и дешифрования данных в криптографической системе с открытыми ключами RSA.

2. Постановка задачи

Разработать программу генерации ключей по p и q, выбора минимального допустимого e, проверки взаимной простоты, шифрования и дешифрования текста, включая русский алфавит. Вывести ключи и результаты; провести тестирование.

3. Исходные данные и расчёты

Номер в списке: 30. Число вариантов: 14. 30 mod 14 = 2. По таблице 4.1: p = 211, q = 317. Оба числа простые и различны. n = p·q = 66887. φ(n) = (211−1)(317−1) = 66360. Минимальное e > 1 с НОД(e,66360)=1: e = 11 (2, 3, 5, 7, 9 и 10 не подходят; 4, 6 и 8 также не подходят). d = 24131. Проверка: 11·24131 = 265441 = 4·66360 + 1. Открытый ключ (11, 66887), закрытый ключ (24131, 66887).

4. Алгоритм программы

1) Проверить простоту и различие p и q. 2) Вычислить n и φ(n). 3) Найти минимальное e с НОД(e,φ(n))=1. 4) Расширенным алгоритмом Евклида вычислить d. 5) Преобразовать сообщение в байты UTF-8. 6) Для каждого байта m вычислить c = m^e mod n. 7) Для каждого c вычислить m = c^d mod n. 8) Декодировать UTF-8 и сравнить тексты.

5. Текст программы

```
#!/usr/bin/env python3
"""Лабораторная работа № 2. RSA. Вариант 2."""
from math import gcd, isqrt
P, Q = 211, 317
def is_prime(number):
    if number < 2:
        return False
    for divisor in range(2, isqrt(number) + 1):
        if number % divisor == 0:
            return False
    return True
def are_coprime(a, b):
    return gcd(a, b) == 1
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    divisor, x1, y1 = extended_gcd(b, a % b)
    return divisor, y1, x1 - (a // b) * y1
def modular_inverse(e, phi):
    divisor, x, _ = extended_gcd(e, phi)
    if divisor != 1:
        raise ValueError("Обратный элемент не существует")
    return x % phi
def generate_keys(p, q):
    if not is_prime(p) or not is_prime(q):
        raise ValueError("p и q должны быть простыми")
    if p == q:
        raise ValueError("p и q должны различаться")
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 2
    while e < phi and not are_coprime(e, phi):
        e += 1
    if e >= phi:
        raise ValueError("Не найдено допустимое e")
    d = modular_inverse(e, phi)
    return (e, n), (d, n), phi
```

```
def encrypt(message, public_key):
    e, n = public_key
    if n <= 255:
        raise ValueError("Модуль должен быть больше 255")
    return [pow(byte, e, n) for byte in message.encode("utf-8")]
def decrypt(encrypted, private_key):
    d, n = private_key
    result = []
    for number in encrypted:
        if not isinstance(number, int) or not 0 <= number < n:
            raise ValueError("Некорректный блок шифротекста")
        byte = pow(number, d, n)
        if not 0 <= byte <= 255:
            raise ValueError("Результат не является байтом")
        result.append(byte)
    return bytes(result).decode("utf-8")
def main():
    public_key, private_key, phi = generate_keys(P, Q)
    e, n = public_key
    d, _ = private_key
    print("Лабораторная работа №2. RSA. Вариант №2")
    print(f"p={P}, q={Q}, n={n}, phi(n)={phi}, e={e}, d={d}")
    print(f"НОД(e, phi)={gcd(e, phi)}")
    print(f"(e*d) mod phi={(e*d)%phi}")
    print(f"Открытый ключ: {public_key}")
    print(f"Закрытый ключ: {private_key}")
    message = input("Введите сообщение: ")
    encrypted = encrypt(message, public_key)
    print(f"Исходное сообщение: {message}")
    print(f"Зашифрованное сообщение: {encrypted}")
    decrypted = decrypt(encrypted, private_key)
    print(f"Расшифрованное сообщение: {decrypted}")
    print("УСПЕШНО" if message == decrypted else "ОШИБКА")
if __name__ == "__main__":
    main()
```

5. Результаты выполнения

На предоставленном скриншоте программы использовано сообщение «Привет, RSA! Илья 30 номер в группе». Программа вывела ключи (11, 66887) и (24131, 66887), числовой шифротекст и исходное сообщение после расшифрования. Отображено «УСПЕШНО: исходное сообщение восстановлено». На предоставленном скриншоте тестирования: Ran 10 tests in 0.000s; OK.

6. Анализ результатов

Вычисленные значения ключей удовлетворяют условиям RSA. Дешифрование восстановило исходную строку, в том числе кириллицу, латиницу, пробелы и цифры. Десять автоматических тестов прошли успешно; дополнительно повторный запуск тестов приложенного кода выполнен при подготовке отчёта. Каждый байт UTF-8 шифруется отдельно, поскольку $0 \le m \le 255 < n$. Учебная схема является детерминированной, использует очень малый модуль и не применяет OAEP, поэтому криптографически небезопасна для реальных данных.

Рисунок 1 — Выполнение программы RSA

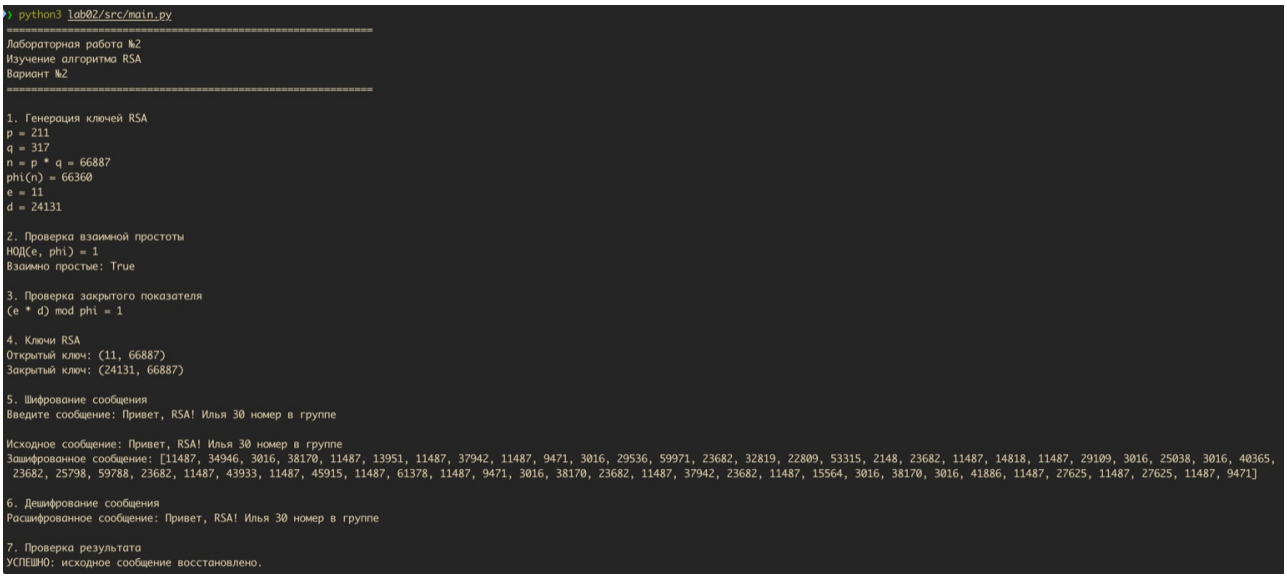
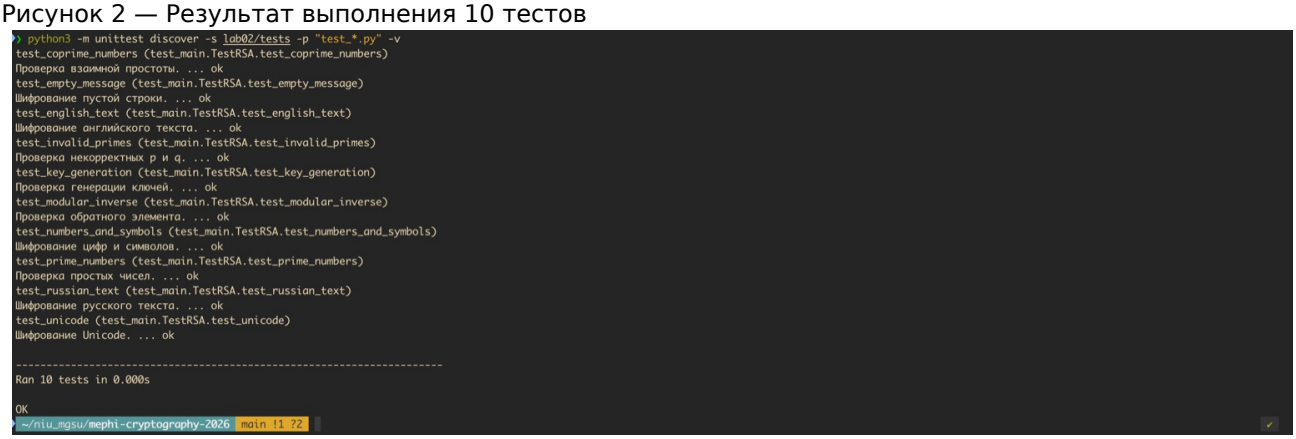


Рисунок 2 — Результат выполнения 10 тестов



7. Ответы на контрольные вопросы

1. Что такое однонаправленные функции?

Функции, для которых вычисление значения по аргументу эффективно, а восстановление аргумента по результату вычислительно трудно.

2. Основные свойства однонаправленных функций с потайным ходом.

Без секретной дополнительной информации обратное вычисление трудно; при наличии «потайного хода» оно выполняется эффективно.

3. Какие числа называются взаимно простыми?

Два целых числа взаимно просты, если их наибольший общий делитель равен 1.

4. Как реализуется программное возведение в степень для больших чисел?

Применяют быстрое модульное возведение в степень (двоичное возведение в степень), последовательно сокращая промежуточные результаты по модулю. В Python используется pow(a, b, n).

5. На чем основана криптостойкость алгоритма RSA?

На вычислительной сложности восстановления секретного ключа по открытому; для классического RSA важна трудность факторизации большого модуля n. Малые учебные параметры небезопасны.

6. Каковы достоинства и недостатки асимметричных алгоритмов?

Преимущества: не нужен общий секретный ключ для шифрования и возможны цифровые подписи. Недостатки: сравнительно низкая скорость, большие ключи, необходимость проверки подлинности открытого ключа.

7. В чем заключается основная проблема симметричных криптосистем?

В безопасной передаче и распределении общего секретного ключа между участниками.

8. Как вычисляется функция Эйлера φ(n) для модуля n = p·q?

Если p и q — различные простые числа, $\varphi(n) = (p-1)(q-1)$. В данном варианте $\varphi(n) = 210 \cdot 316 = 66360$.

9. Какие требования предъявляются к числу e при генерации открытого ключа?

Нужно $1 < e < \varphi(n)$ и $\text{НОД}(e, \varphi(n)) = 1$. По заданию выбирается минимальное допустимое e = 11.

10. Как найти число d для закрытого ключа, зная e и φ(n)?

Найти обратный по модулю элемент: $e \cdot d \equiv 1 \pmod{\varphi(n)}$; используется расширенный алгоритм Евклида. Здесь d = 24131.

11. Почему безопасность RSA зависит от сложности факторизации числа n?

Зная множители p и q, можно вычислить φ(n) и восстановить d. Для больших правильно выбранных простых чисел факторизация n практически сложна.

12. Какие математические свойства обеспечивают корректность дешифрования в RSA?

Соотношение $ed \equiv 1 \pmod{\varphi(n)}$, теорема Эйлера для взаимно простых с n сообщений и малая теорема Ферма/китайская теорема об остатках для общего случая.

13. Что произойдет, если выбрать e, не взаимно простое с φ(n)?

Мультипликативного обратного элемента d по модулю φ(n) не будет; стандартная генерация ключей RSA невозможна.

14. Почему при выборе p и q из таблицы важно, чтобы они были простыми?

Формула $\varphi(pq) = (p-1)(q-1)$ для различных простых p, q обеспечивает правильный расчёт ключей и корректность математической схемы.

15. Объясните роль теоремы Эйлера в алгоритме RSA.

Для НОД(m,n)=1 справедливо m^φ(n) ≡ 1 (mod n). Поскольку ed = 1 + kφ(n), получаем m^(ed) ≡ m (mod n).

8. Выводы

Реализованы генерация ключей RSA, проверка взаимной простоты, шифрование и дешифрование UTF-8 текста. Для варианта 2 получены e=11, d=24131, n=66887. Успешная расшифровка и 10 пройденных тестов подтверждают корректность учебной реализации в проверенных сценариях.